

ARM PrimeCell Driver

VIC v1.0 (PL192)
Test Report

PrimeCell Driver Test Report

© Copyright ARM Limited 2001. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is Open Access. This document has no restriction on distribution.

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Document Summary

Document Issue	B
Primecell driver identifier	VIC
Code Version	1.0
Primecell identifier	PL192
Test date	23, October, 2002

Contents

1	ABOUT THIS DOCUMENT	5
1.1	Change History	5
2	RELEASED FILES	5
3	TEST ENVIRONMENT	5
3.1	Test Specification Summary	5
3.2	Hardware Requirements	7
4	CODE COVERAGE	8
4.1	Overall	8
4.2	Untested Blocks	8
4.3	Untested Routine Exits	9
5	RELEASED TEST MODULE	11
5.1	Overview	11
5.2	Changes to Test Environment	11
5.3	Outline	11
5.4	Initialisation	11
5.5	Reading Interrupt Status	11
5.6	Sequential Interrupts Test	11
5.7	Re-entrant Interrupts Test	13
5.8	Priority Level Masking Test	15
5.9	Global Sequential Interrupts Test	17
5.10	Vector Priority Daisy Chain Test	18
5.11	Test Completion	20
5.12	Results	20

1 ABOUT THIS DOCUMENT

1.1 Change History

Issue	Date	Change
A	26, September, 2002	First release of test report
B	23, October 2002	Next release of test report

2 RELEASED FILES

PL192-ISPC-1000-F	API document
PL192-TREP-1000-B	This document
appl192/appl192.h	Public header file
appl192/pl192.h	Private header file
appl192/pl192wrappers.s	Driver source code
appl192/pl192dispatch.s	Driver source code
appl192/pl192.c	Driver source code
appl192/pl192tst.c	Released test code
appl192/tstpl192.s	Released test code

3 TEST ENVIRONMENT

3.1 Test Specification Summary

	Specification (Intended)	Actual Test Environment If different from Specification
Code Test Platform	INTEGRATOR	
Core used for test	ARM940T	
Module ROM size (RO code + RO data)	3920	
Module RAM size (ZI data) per instance	1512	
Primecell tested	PL192	
External hardware used	N/A	
Performance obtained	N/A	

Driver modules	pl192.c, pl192wrappers.s, pl192dispatch.s	
Test modules	pl192tst.c, tstpl192.s	
Standard modules required (normal archive location)	os.c, oss.s, int.c, inttst.c, dispatch.s, boot.s, vectors.s, test.c, main.c	
Custom modules required (in [project]/privates/ads)	applatfm.h	

3.2 Hardware Requirements

3.2.1 FPGA

Is the image used to program the FPGA archived	NO
If so, at what location	N/A

3.2.2 Test configuration

Is the hardware configuration documented	NO
If so, at what location	N/A

FPGA setup:

VIC #0, VIC #1, Trick box for VIC #0 (not used), GPIO (not used), Watchdog (not used), Timer 1 and 2 (not used) and Interrupt generator (not used).

Base addresses:

VIC #0	0xC0000000
VIC #1	0xC1000000
Trick box for VIC #0	0xC2000000
GPIO	0xC4000000
Watchdog	0xC4100000
Timer 1 and 2	0xC4200000
GPIO inputs	0xC4300000
Interrupt generator	0xC4400000

3.2.3 External connectors required

VIC #0 IRQ is connected to the EXPINT0 input of the Integrator.

VIC #0 FIQ is connected to the EXPINT1 input of the Integrator.

3.2.4 Devices on prototyping area

none

3.2.5 External test equipment

none

4 CODE COVERAGE

4.1 Overall

Code coverage tests have been performed (generated by v1.18) **YES**

Code coverage details below

- ◆ *Clear the data area below the application base address.*
- ◆ *Run all test applications.*

Percentage of normal routine exits tested. 50%

- ◆ *Examine all routine exits listed as untested to ensure that these are not used in normal flow.*

All untested routine exits examined. **YES**

Percentage of blocks tested. 68%

4.2 Untested Blocks

Blocks not covered by these tests have been examined and categorised as below.

Block number	Routine	Reason
1	apVIC_Initialize	This only happens when an invalid VIC number parameter is being sought
3	apVIC_Initialize	This part of code is not compiled when apOS_CONFIG_VIC_0_BLOCKING_MODE is not defined
9	apVIC_PrioritySet	This only happens if the function is called inside a handler, which is invalid
10	apVIC_PrioritySet	This only happens when an invalid source number is being sought
11	apVIC_PrioritySet	This only happens when there was an attempt to change interrupt type of Raw Interrupt source, which is invalid
12	apVIC_PrioritySet	This only happens when an invalid priority parameter is being sought
14	apVIC_PriorityGet	This only happens when an invalid source number is being sought
16	apVIC_PriorityMaskSet	This only happens when an invalid VIC number parameter is being sought
18	apVIC_PriorityMaskGet	This only happens when an invalid VIC number parameter is being sought
20	apVIC_PriorityDaisySet	This only happens when an invalid VIC number parameter is being sought
22	apVIC_PriorityDaisySet	This part of code is compiled only when apOS_CONFIG_VIC_0_BLOCKING_MODE is defined, and the

		function is called inside a handler with an attempt to change priority to the higher value, which is invalid
24	apVIC_PriorityDaisyGet	This only happens when an invalid VIC number parameter is being sought
19, 20	apVIC_InterruptEnable	This only happens when an invalid source number is being enabled
36	apVIC_HandlerSet	This only happens if the function is called inside a handler, which is invalid
37	apVIC_HandlerSet	This only happens when an invalid source number is being sought
39	apVIC_RawHandlerSet	This only happens if the function is called inside a handler, which is invalid
40	apVIC_RawHandlerSet	This only happens when an invalid source number is being sought
42	apVIC_HandlerClear	This only happens when an invalid source number is being sought
48	apVIC_SingleFIQSourceSet	This part of code is compiled only when apOS_CONFIG_VIC_SINGLE_FIQ is defined
49	apVIC_SingleFIQSourceSet	This part of code is compiled only when apOS_CONFIG_VIC_SINGLE_FIQ is defined, and the function is called inside a handler, which is invalid
50	apVIC_SingleFIQSourceSet	This part of code is compiled only when apOS_CONFIG_VIC_SINGLE_FIQ is defined, and an invalid source number is being sought
51	apVIC_SingleFIQSourceSet	This part of code is compiled only when apOS_CONFIG_VIC_SINGLE_FIQ is defined, and an interrupt source type is not FIQ, which is invalid
52	apVIC_SingleFIQSourceClear	This part of code is compiled only when apOS_CONFIG_VIC_SINGLE_FIQ is defined
55	VIC_DefaultHandler	This can not be reached as long as the handler is set up correctly
58, 61	VIC_FindSource	This only happens when an invalid source number is being sought
62	VIC_FindEntry	This only happens when an invalid source number is being sought
74	VIC_GenerateTables	This only happens when an invalid number of interrupt sources exceeding the limit is being sought
78	VIC_GenerateTables	Code never executed – included for safety.

4.3 Untested Routine Exits

Exit number	Routine	Reason
1	apVIC_Initialize	This only happens when an invalid VIC number parameter is being sought
2	apVIC_PrioritySet	This only happens if the function is called inside a handler, which is

		invalid
3	apVIC_PrioritySet	This only happens when an invalid source number is being sought
4	apVIC_PrioritySet	This only happens when there was an attempt to change interrupt type of Raw Interrupt source, which is invalid
5	apVIC_PrioritySet	This only happens when an invalid priority parameter is being sought
7	apVIC_PriorityGet	This only happens when an invalid source number is being sought
9	apVIC_PriorityMaskSet	This only happens when an invalid VIC number parameter is being sought
11	apVIC_PriorityMaskGet	This only happens when an invalid VIC number parameter is being sought
13	apVIC_PriorityDaisySet	This only happens when an invalid VIC number parameter is being sought
15	apVIC_PriorityDaisySet	This part of code is compiled only when apOS_CONFIG_VIC_0_BLOCKING_MODE is defined, and the function is called inside a handler with an attempt to change priority to the higher value, which is invalid
17	apVIC_PriorityDaisyGet	This only happens when an invalid VIC number parameter is being sought
27, 28	apVIC_InterruptEnable	This only happens when an invalid source number is being enabled
26	apVIC_HandlerSet	This only happens if the function is called inside a handler, which is invalid
27	apVIC_HandlerSet	This only happens when an invalid source number is being sought
29	apVIC_RawHandlerSet	This only happens if the function is called inside a handler, which is invalid
30	apVIC_RawHandlerSet	This only happens when an invalid source number is being sought
32	apVIC_HandlerClear	This only happens when an invalid source number is being sought
39	apVIC_SingleFIQSourceSet	This part of code is compiled only when apOS_CONFIG_VIC_SINGLE_FIQ is defined, and the function is called inside a handler, which is invalid
40	apVIC_SingleFIQSourceSet	This part of code is compiled only when apOS_CONFIG_VIC_SINGLE_FIQ is defined, and an invalid source number is being sought
41	apVIC_SingleFIQSourceSet	This part of code is compiled only when apOS_CONFIG_VIC_SINGLE_FIQ is defined, and an interrupt source type is not FIQ, which is invalid
42	apVIC_SingleFIQSourceSet	This part of code is compiled only when apOS_CONFIG_VIC_SINGLE_FIQ is defined
43	apVIC_SingleFIQSourceClear	This part of code is compiled only when apOS_CONFIG_VIC_SINGLE_FIQ is defined

46	VIC_DefaultHandler	This can not be reached as long as the handler is set up correctly
50, 51	VIC_FindSource	This only happens when an invalid source number is being sought
53	VIC_FindEntry	This only happens when an invalid source number is being sought

5 RELEASED TEST MODULE

5.1 Overview

The released test modules are named **pl192tst.c** and **tstpl192.s**.

They are used as a self-test module for the VIC PL192 PrimeCell and also as a part of the formal testing.

Test environment is as described earlier **YES**

Code coverage: Percentage of blocks tested by this test module 68%

5.2 Changes to Test Environment

The actual VICs are on the Logic Module. VIC #1 is daisy-chained to VIC #0 IRQ and FIQ outputs of which are connected to the EXPINT1 and EXPINT0 inputs of Interrupt controller on the Integrator.

5.3 Outline

The public test code is intended to exercise the VIC and prove that it is able to generate ARM IRQs and FIQs. The tests are not intended to exercise the entire VIC API. The test executes as normal IRQ and FIQ handling (which used `apVIC_HandlerSet()`) and Raw ISR mode which can speed up the handling response (which used `apVIC_RawHandlerSet()` accordingly).

5.4 Initialisation

Call to `apVIC_Initialize()` function to set up the VIC base address and initial interrupt sources' priority.

- Number of interrupts used **NONE**
- Specific parameters passed in configuration data **NONE**

5.5 Reading Interrupt Status

Call to `apVIC_GetInterruptStatus()` function to define status of interrupt sources. Asserted interrupts then are excluded from further tests.

5.6 Sequential Interrupts Test

This is done by sequence of calls `VIC_TestReEntrant()` for IRQ vectored and FIQ non-vectored interrupts, `VIC_TestRawReEntrant()` for Raw interrupts and `VIC_GeneralTest()` for all types of interrupts, but with macro `apOS_CONFIG_INT_REENTRANT` set to 0 in **apintcfg.h**. The first two functions call `apVIC_PriorityDaisySet()`, which sets up the lowest (15) priority value for daisy-chained interrupts and then set up 32 interrupt sources in VIC

with the priority from high to low (0 to 15 for IRQ vectored and from 0 to 31 for FIQ non-vectored). Then the public functions calling sequence is:

- apVIC_InterruptDisable();
- apVIC_HandlerSet() or apVIC_RawHandlerSet();
- apVIC_PrioritySet();
- apVIC_InterruptEnable();

Above calling sequence will be repeated for 32 times with all 32 interrupt sources being set up. In this test, all of the IRQ vectored and FIQ non-vectored interrupts will be handled by the same interrupt handler. Each Raw interrupt has its own interrupt handler. And then use public function call

- apVIC_ProgrammedInterruptSet()

to generate a software interrupt which is set to the lowest priority in the earlier calling sequence. Upon entering the interrupt handler, a new software interrupt with one level higher priority than the current one is generated. However, since we are performing sequential interrupts test, no new interrupt can be handled until the current one is finished. When entering the handler in the last time, flags are set up to indicate all interrupts are handled correctly.

5.6.1 Detailed Description

The test code does the following:

- ◆ Initialises the interrupt sources. Each interrupt source is:
 - disabled
 - assigned an interrupt handler
 - configured to raise an IRQ or FIQ
 - assigned a priority (such that the first interrupt source 0 is the highest priority, and the last 31, is the lowest)
 - re-enabled.
- ◆ Raises a soft interrupt on the lowest priority interrupt source – 31.
- ◆ Waits for the test to complete or time out waiting (5 s) and reports the result.

For each interrupt, the interrupt handler should be called by the mini-dispatcher implemented in **pl192wrappers.s** (IRQ vectored and Raw interrupts) and **pl192dispatch.s** (non-vectored FIQ) and will take the following actions:

- ◆ Check that this is the source that should be raising an interrupt (on the correct VIC).
- ◆ Check that this is the first time the interrupt source has interrupted.
- ◆ Disable the interrupt source at the VIC.
- ◆ If this is not the last interrupt source, then:
 - updates an internal variable that indicates which source should next interrupt such that it is set to the next one level higher priority interrupt source
 - raises a soft interrupt on that source.
- ◆ When the last interrupt source has interrupted, the test is complete.
- ◆ Before exiting, the interrupt handler does the following:
 - clears the soft interrupt
 - re-enables the interrupt source.

5.6.2 Results

The possible results for this test are as follows:

If any interrupt occurs for the wrong source or for the wrong VIC, then the test code will display the following message and terminate the test:

`Testing Vectored interrupts: Fail`

If the test times out due to any interrupt not occurring, then the following message will be displayed:

`Interrupt test: Sequential Interrupts - Test Failed to complete: Fail`

If the test code deduces that all interrupts have occurred correctly and were handled sequentially, then the following message will be displayed when the test has completed:

`Testing Vectored interrupts: Pass`

The following is the standard output from the Sequential Interrupts Test:

- -----
- Testing module VIC at 0xc0000000
- Testing Vectored interrupts: Pass
- Testing Raw interrupts: Pass
- Testing FIQ interrupts: Pass
- Testing all interrupt sources: Pass

5.7 Re-entrant Interrupts Test

This is done by sequence of calls `VIC_TestReEntrant()` for IRQ vectored and FIQ non-vectored interrupts, `VIC_TestRawReEntrant()` for Raw interrupts and `VIC_GeneralTest()` for all types of interrupts, but with macro `apOS_CONFIG_INT_REENTRANT` set to 1 in **apintcfg.h**. Because of VIC PL192 hardware implementation the number of re-entrant interrupts in the system is limited to 16. The first two functions call `apVIC_PriorityDaisySet()`, which sets up the lowest (15) priority value for daisy-chained interrupts and then sets up 16 interrupts in VIC with the priority from high to low (0 to 15) for IRQ vectored and 32 interrupts (priority from 0 to 31) for FIQ non-vectored. The last function performs the test. In VIC 0 blocking mode (macro `apOS_CONFIG_VIC_0_BLOCKING_MODE` set to 1 in **apintcfg.h**). The public functions calling sequence is:

- `apVIC_InterruptDisable();`
- `apVIC_HandlerSet()` or `apVIC_RawHandlerSet();`
- `apVIC_PrioritySet();`
- `apVIC_InterruptEnable();`

Above calling sequence will be repeated for 16 or 32 times with all 16 or 32 interrupt sources being set up. In this test, all of the IRQ vectored and FIQ non-vectored interrupts will be handled by the same interrupt handler. Each Raw interrupt has its own interrupt handler. And then use public function call

- `apVIC_ProgrammedInterruptSet()`

to generate a software interrupt which is set to the lowest priority in the earlier calling sequence. Upon entering the interrupt handler a new software interrupt with one level higher priority than the current one is generated. So the execution flow turns to service the higher priority interrupt which will then generate an interrupt with further one level higher priority, and so on. In such a way, all interrupts, except the one with the highest priority but is generated last, are handled re-entrantly. When entering the handler in the last time, flags are set up to indicate all interrupts are handled correctly.

5.7.1 Detailed Description

The test code does the following:

- ◆ Initialises the interrupt sources. Each interrupt source is:
 - disabled
 - assigned an interrupt handler
 - configured to raise an IRQ or FIQ
 - assigned a priority (such that the first interrupt source 0 is the highest priority, and the last 15 or 31, is the lowest), for all daisy-chained sources the priority of servicing interrupts is defined by the value set up by `apVIC_PriorityDaisySet()` function
 - re-enabled.
- ◆ Raises a soft interrupt on the lowest priority interrupt source – 15 or 31.
- ◆ Waits for the test to complete or time out waiting (5 s) and reports the result.

For each interrupt, the interrupt handler should be called by the mini-dispatcher implemented in **pl192wrappers.s** (IRQ vectored and Raw interrupts) and **pl192dispatch.s** (non-vectored FIQ) and will take the following actions:

- ◆ Check that this is the source that should be raising an interrupt (on the correct VIC).
- ◆ Check that this is the first time the interrupt source has interrupted.
- ◆ Disable the interrupt source at the VIC.
- ◆ For daisy-chained interrupts set up new value of daisy priority with one level higher than the current one.
- ◆ If this is not the last interrupt source, then:
 - enables Core IRQ and FIQ for FIQ re-entrant handler (for IRQ vectored interrupts it is done by mini dispatcher)
 - updates an internal variable that indicates which source should next interrupt such that it is set to the next one level higher priority interrupt source
 - raises a soft interrupt on that source.
- ◆ When the last interrupt source has interrupted, the test is complete.
- ◆ Before exiting, the interrupt handler does the following:
 - clears the soft interrupt
 - re-enables the interrupt source.

5.7.2 Results

The possible results for this test are as follows:

If any interrupt occurs for the wrong source or for the wrong VIC, then the test code will display the following message and terminate the test:

`Testing Vectored interrupts: Fail`

If the test times out due to any interrupt not occurring, then the following message will be displayed:

`Interrupt test: Re-entrant Interrupts - Test Failed to complete: Fail`

If the test code deduces that the VIC dispatcher was built to support re-entrant / nested interrupts but the interrupts were not serviced re-entrantly, then the following message will be displayed when the test has completed:

`Interrupt test: Re-entrant interrupt test failed but
only because the interrupts were handled sequentially.`

If the test code deduces that the all interrupts have occurred correctly and were handled re-entrantly, then the following message will be displayed when the test has completed:

`Testing Vectored interrupts: Pass`

The following is the standard output from the Sequential Interrupts Test:

- -----
- Testing module VIC at 0xc0000000
- Testing Vectored interrupts: Pass
- Testing Raw interrupts: Pass
- Testing FIQ interrupts: Pass
- Testing all interrupt sources: Pass

5.8 Priority Level Masking Test

This is done by call VIC_TestMaskPriority() for specified interrupt controller and the public functions calling sequence is:

- apVIC_InterruptDisable();
- apVIC_HandlerSet();
- apVIC_PrioritySet();

Above calling sequence will be repeated for 32 times with all 32 IRQ vectored interrupt sources being set up. In this test, all of the IRQ vectored interrupts will be handled by the same interrupt. Then use public function calls

- apVIC_PriorityMaskSet();
- apVIC_PriorityMaskGet();
- apVIC_InterruptEnable();
- apVIC_ProgrammedInterruptSet();
- apVIC_InterruptDisable();
- apVIC_ProgrammedInterruptClear();

to set priority level masks and generate a software interrupts on all interrupt sources.

5.8.1 Detailed Description

The test code does the following:

- ◆ Initialises the interrupt sources. Each interrupt source is:
 - disabled
 - assigned an interrupt handler
 - configured to raise an IRQ
 - assigned a priority, which is used for priority level masking, such that the first interrupt source 0 is the highest priority (0), and the last 31 is the lowest (15),
 - re-enabled.
- ◆ Clears the interrupt occurrence flag.
- ◆ Sets up the priority level mask.
- ◆ Reads the current priority level mask and compares it with the set up value and returns an error if they do not match.
- ◆ Raises a soft interrupt on all interrupt sources.
- ◆ Waits for the test to complete or time out waiting (50 ms).
- ◆ Disables all interrupt sources.
- ◆ Clears all software interrupts.
- ◆ Checks the interrupt occurrence flag for the correct value.

The above sequence is repeated for all interrupt sources with different values of the priority level mask changing from 0xFFFE to 0x7FFF.

For each interrupt, the interrupt handler should be called by the mini-dispatcher implemented in `pl192wrappers.s` and will take the following actions:

- ◆ Check that this is the source that should be raising an interrupt (on the correct VIC).
- ◆ Check that this is the first time the interrupt source has interrupted.
- ◆ Disable the interrupt source at the VIC.
- ◆ Updates the interrupt occurrence flag.
- ◆ Clears the software interrupt.
- ◆ Before exiting, the interrupt handler re-enables the interrupt source.

5.8.2 Results

The possible results for this test are as follows:

If any interrupt occurs for the wrong source or for the wrong VIC, then the test code will display the following message and terminate the test:

`Testing masking priority levels: Fail`

If the test code deduces that all interrupts have occurred correctly, then the following message will be displayed when the test has completed:

`Testing masking priority levels: Pass`

The following is the standard output from the Priority Level Masking Test:

- Testing masking priority levels: Pass

5.9 Global Sequential Interrupts Test

This is done by sequence of calls VIC_GlobalTest() for IRQ vectored and Raw interrupts on both interrupt controllers. The test sets up 32 interrupt sources in VIC #0 and VIC #1 with the priority from high to low (0 to 15). The public functions calling sequence is:

- apVIC_InterruptDisable();
- apVIC_HandlerSet() or apVIC_RawHandlerSet();
- apVIC_PrioritySet();
- apVIC_InterruptEnable();

Above calling sequence will be repeated for 64 times with all 64 interrupt sources being set up. In this test, all of the IRQ vectored interrupts will be handled by the same interrupt handler. Each Raw interrupt has its own interrupt handler. And then use public function call

- apVIC_ProgrammedInterruptSet()

to generate a software interrupt on VIC #1 which is set to the lowest priority in the earlier calling sequence. Upon entering the interrupt handler, a new software interrupt with one level higher priority than the current one is generated. However, since we are performing sequential interrupts test, no new interrupt can be handled until the current one is finished. When entering the handler in the last time, flags are set up to indicate all interrupts are handled correctly.

5.9.1 Detailed Description

The test code does the following:

- ◆ Initialises the interrupt sources. Each interrupt source is:
 - disabled
 - assigned an interrupt handler
 - configured to raise an IRQ
 - assigned a priority, such that the first interrupt source 0 is the highest priority (0), and the last 31, is the lowest (15)
 - re-enabled.
- ◆ Raises a soft interrupt on the lowest priority interrupt source 31 on VIC #1.
- ◆ Waits for the test to complete or time out waiting (5 s) and reports the result.

For each interrupt, the interrupt handler should be called by the mini-dispatcher implemented in **pl192wrappers.s** and will take the following actions:

- ◆ Check that this is the source that should be raising an interrupt (on the correct VIC).
- ◆ Check that this is the first time the interrupt source has interrupted.
- ◆ Disable the interrupt source at the VIC.
- ◆ If this is not the last interrupt source 0 on VIC #0, then:
 - updates an internal variable that indicates which source should next interrupt such that it is set to the next one level higher priority interrupt source
 - raises a soft interrupt on that source.
- ◆ When the last interrupt source 0 on VIC #0 has interrupted, the test is complete.

-
- ◆ Before exiting, the interrupt handler does the following:
 - clears the soft interrupt
 - re-enables the interrupt source.

5.9.2 Results

The possible results for this test are as follows:

If any interrupt occurs for the wrong source or for the wrong VIC, then the test code will display the following message and terminate the test:

`Testing all interrupts sources on both VICs: Fail`

If the test times out due to any interrupt not occurring, then the following message will be displayed:

`Interrupt test: Sequential Interrupts - Test Failed to complete: Fail`

If the test code deduces that all interrupts have occurred correctly and were handled sequentially, then the following message will be displayed when the test has completed:

`Testing all interrupts sources on both VICs: Pass`

The following is the standard output from the Sequential Interrupts Test:

- Testing all interrupts sources on both VICs: Pass

5.10 Vector Priority Daisy Chain Test

This is done by call VIC_TestDaisyPriority() and the public functions calling sequence is:

- apVIC_InterruptDisable();
- apVIC_HandlerSet();
- apVIC_PrioritySet();

Above calling sequence will be repeated for 32 times with all 32 IRQ vectored interrupt sources being set up. In this test, all of the IRQ vectored interrupts will be handled by the same interrupt. Then use public function calls

- apVIC_PriorityDaisySet();
- apVIC_PriorityDaisyGet();
- apVIC_PriorityMaskSet();
- apVIC_PriorityMaskGet();
- apVIC_InterruptEnable();
- apVIC_ProgrammedInterruptSet();
- apVIC_InterruptDisable();
- apVIC_ProgrammedInterruptClear();

to set priority daisy and priority level masks and generate a software interrupts on all interrupt sources.

5.10.1 Detailed Description

The test code does the following:

- ◆ Initialises the daisy-chained interrupt sources. Each interrupt source is:
 - disabled
 - assigned an interrupt handler
 - configured to raise an IRQ
 - assigned a priority, such that the first interrupt source 0 is the highest priority (0), and the last 31 is the lowest (15)
 - re-enabled.
- ◆ Sets up the daisy priority.
- ◆ Reads the current daisy priority and compares it with the set up value and returns an error if they do not match.
- ◆ Sets up or clears the priority level mask.
- ◆ Reads the current priority level mask and compares it with the set up value and returns an error if they do not match.
- ◆ Clears the interrupt occurrence flag.
- ◆ Raises a soft interrupt on all interrupt sources.
- ◆ Waits for the test to complete or time out waiting (50 ms).
- ◆ Disables all interrupt sources.
- ◆ Clears all software interrupts.
- ◆ Checks the interrupt occurrence flag for the correct value.

The above sequence is repeated for all daisy-chained interrupt sources with different values of daisy priority, changing from 15 to 0, and the priority level mask from 0x7FFF to 0xFFFFE and from 0x8000 to 0x0001.

For each interrupt, the interrupt handler should be called by the mini-dispatcher implemented in `pl192wrappers.s` and will take the following actions:

- ◆ Check that this is the source that should be raising an interrupt (on the correct VIC).
- ◆ Check that this is the first time the interrupt source has interrupted.
- ◆ Disable the interrupt source at the VIC.
- ◆ Updates the interrupt occurrence flag.
- ◆ Clears the software interrupt.
- ◆ Before exiting, the interrupt handler re-enables the interrupt source.

5.10.2 Results

The possible results for this test are as follows:

If any interrupt occurs for the wrong source or for the wrong VIC, then the test code will display the following message and terminate the test:

Testing Vector Priority Daisy: Fail

If the test code deduces that all interrupts have occurred correctly, then the following message will be displayed when the test has completed:

Testing Vector Priority Daisy: Pass

The following is the standard output from the Vector Priority Daisy Chain Test:

- Testing Vector Priority Daisy: Pass

5.11 Test Completion

When all tests have completed, the test code performs the following 'cleanup' actions to ensure the VICs are left in a suitable state for use by other modules:

- Clears the handler for all 64 interrupt sources by calling `apVIC_HandlerClear()` – which will also disable them.

5.12 Results

Test run returns TRUE

YES

When **apTEST_INTERACTIVE** is FALSE, no user interaction is required

N/A

When **apTEST_VERBOSE** is FALSE, no output is produced

NO